CASE-STUDY PRACTICE | CAPSTONE 15 | STUDY EDITION

LLM API Gateway

Scenario questions on a gateway that fronts several LLM providers, each opening into deeper follow-ups, with answers that explain how to think. Built on the AI System Design Primer case study of the same name.

Jump to the contents | The case in one page | Numbers to know | Decision table | Sources

HOW TO STUDY WITH THIS BOOK

1. Read the amber **concepts box** at the start of a part and look up anything unfamiliar. 2. Read a **scenario** and, before reading further, write down what you would ask and what you would check. 3. Read the green **how to think** box, which names the mental move the question tests. 4. Walk down the nested steps: cover each answer, predict it, then compare. Each answer exposes the next question. The last step of every question asks for the general lesson in one sentence.

The pattern behind most questions

- A number or a technique arrives with authority: a published benchmark, a cost estimate, a shard count, a rate.
- The interviewer asks what it assumes, and what happens when the assumption fails.
- A strong answer names the hidden assumption, puts a number on it, and says how to measure it on your own data.
- The last step usually reframes the original question.

EVIDENCE TAGS

illustrative: this document's planning assumption. **recomputed**: arithmetic re-derived from the stated inputs. **docs, 2026-10-17**: read from the products' documentation pages on that date. **reasoned**: an argument from the design, not tested.

A NOTE ON SOURCES AND ON WHAT WAS CHECKED

The scenarios are written for this document; none comes from a real company question bank. Nothing here was run against a gateway, a provider or a counter store: product behavior is taken from the products' documentation, and every figure is either illustrative or arithmetic from stated inputs. The last page lists the sources and says how much of each was read. A separate Mock Interview edition (answers hidden) uses the same questions and numbering.

Contents

Every line below is a link: click it to jump to that page. The outline panel of your PDF viewer lists the same entries, and the “Contents” link in every page footer brings you back here.

Front matter

A The case in one page 3

B Numbers to know 4

C Decision table 5

Parts at a glance

1 Framing the problem: tokens, money and admission 6

2 Token buckets and limits 11

3 Routing, fallback and caching 16

4 Architecture, failure modes and safety 22

Sources and confidence 27

Every question, by part

Part 1 Framing the problem: tokens, money and admission 6

01 The request limit says 100%, finance says 130% Framing 6

02 A token budget, or a dollar budget? Design choice 7

03 Admit a request before you know the answer Mechanism 8

04 Is token counting too slow for a 20 ms budget? Latency 9

Part 2 Token buckets and limits 11

05 Why not just reset a counter every minute? Mechanism 11

06 Review the multi-scope limiter Code review 12

07 Size the bucket Sizing 13

08 Four gateway instances, one limit Scale 14

Part 3 Routing, fallback and caching 16

09 Pick a routing strategy Design choice 16

10 A provider keeps flapping Fallback 17

11 Which errors should the gateway retry? Reliability 18

12 The semantic cache answered the wrong question Caching 19

13 The hit rate collapsed after a release Debugging 20

Part 4 Architecture, failure modes and safety 22

14 Does adding a gateway improve availability? Availability 22

15 The counter store is down Failure mode 23

16 Streaming and the budget Mechanism 24

17 Traffic grows tenfold Scale 25

18 Security review of the gateway Security 26

The case in one page

A summary of the case study to read before the questions.

Problem	One API in front of several LLM providers and models. It authenticates callers, enforces rate limits and budgets, caches responses, routes each request, falls back when a provider fails and meters usage.
Core idea	Count tokens, not requests. Limit the rate in tokens and cap the spend in money, because token prices differ across models. Admit a request on an estimate, then reconcile with the usage the provider reports.
Targets	Gateway-added latency under 20 ms against a model first-token time of 200 ms to 2 s. Gateway availability at least as high as the provider set behind it. All illustrative.
Headline example	490 requests of 50 tokens plus 10 of 128,000 tokens is 500 requests: 100% of a 500-request quota and 130% of a 1,000,000-token budget.
What the products document	LiteLLM: token and request limits and dollar budgets. Portkey: limits on requests or tokens, budgets by cost or tokens. Cloudflare: rate limiting that counts requests (docs, 2026-10-17).
Easy-to-miss traps	Output tokens are unknown at admission. A token budget is not a dollar budget across providers. A bucket as large as the daily budget lets 2 times through in a rolling day. A cache shared across tenants leaks answers. Retries multiply load. A budget store that fails open stops capping.

Numbers to know

Each figure carries the tag that says how it was obtained. Traffic volumes, prices and availabilities are illustrative.

Quantity	Value	Evidence
Request versus token accounting	490 × 50 + 10 × 128,000 = 1,304,500 tokens: 100% of 500 requests, 130% of 1,000,000 tokens	illustrative; recomputed
Reserve and reconcile	2,000 input + 4,096 max_tokens = 6,096 reserved; 2,300 used; 3,796 refunded	illustrative; recomputed
Reservation cost	100 in flight: 609,600 reserved vs 230,000 used (2.65 times); under 100,000 TPM, 16 concurrent vs 43	recomputed
Tokenizing overhead	1 to 2 ms per 2,000 tokens against a 200 ms to 2 s first-token time: 0.05% to 1%	illustrative; recomputed
Bucket refill rates	1,000,000 ÷ 86,400 = 11.57; 5,000,000 ÷ 86,400 = 57.87; 20,000,000 ÷ 86,400 = 231.48 tokens/s	recomputed
Rolling-day worst case	Capacity 1,000,000 + refill 1,000,000 in 24 h = 2,000,000	recomputed
Largest request	128,000 + 4,096 = 132,096 tokens; bucket capacity must be at least this	recomputed
Fixed-window boundary burst	100 per minute: 200 in two seconds; a bucket passes about 103	recomputed
Static split of a limit	100,000 TPM over 4 instances, 60% of demand on one: 65,000 admitted, 35% of demand rejected	illustrative; recomputed
Retry amplification	3 retries per failure: up to 4 times the load; a 10% retry budget: 1.1 times	recomputed
Cache hit rate drop	35% to 5%: provider calls 0.95 ÷ 0.65 = 1.46 times, +46% spend	illustrative; recomputed
Availability	Direct 99.9% (8.76 h/yr); one gateway and one provider 13.1 h; one gateway and two providers 4.4 h; gateway pair and two providers about 39 s	illustrative; recomputed
Counter store down 30 minutes	Runaway client at 100 req/s × 2,300 tokens = 414,000,000 tokens, 414 times a 1,000,000 budget	illustrative; recomputed
Ten times the traffic	CPU 0.4 to 4 cores; counter operations 1,200 to 12,000 a second; provider demand 27.6 to 276 million tokens a minute	illustrative; recomputed
LiteLLM limits and budgets	tpm_limit in tokens per minute, rpm_limit in requests per minute; budgets in US dollars; budgets fail open without a database	docs, 2026-10-17
LiteLLM cooldown and retries	allowed_fails 3, cooldown 5 s, immediate cooldown on 429, more than 50% failures in a minute; no failover for context-window or content-policy errors	docs, 2026-10-17

Decision table

What this design chooses, and when to choose otherwise.

Decision	Default here	Choose otherwise when	Evidence
Rate-limit unit	Tokens per minute plus requests per minute	Requests only if every request costs about the same	docs, 2026-10-17; reasoned
Budget unit	Money per period	Tokens only if one model serves all traffic	docs, 2026-10-17; reasoned
Admission count	Reserve input plus max_tokens, then reconcile	Charge on completion only if overshoot is tolerable	reasoned
Cache scope	Per tenant	Shared only for public prompts	reasoned
Semantic cache	Off for actions and personal data; on for FAQ traffic	On everywhere only with a tested similarity threshold	reasoned
Fallback trigger	Error rate over a minimum sample, plus latency	Latency only if errors are rare	reasoned
Counter store failure	Fail closed or degrade for hard budgets	Fail open for soft limits, with an alert	docs, 2026-10-17; reasoned
Budget overage	Soft overage with alerts for interactive work	Hard cutoff for batch jobs and unknown owners	reasoned

PART 1

Framing the problem: tokens, money and admission

Why a request count is the wrong unit, what to count instead, and how to decide before the answer exists.

READ THESE FIRST: CONCEPTS COVERED IN THIS PART

API gateway	A server that sits in front of backends, so clients talk to one address. It can authenticate, limit traffic, cache, route and log. An LLM gateway is one that fronts several model providers behind a single API.
Token	The unit a language model reads and writes: a piece of a word. Providers bill and limit by tokens, and one request can contain a few tokens or over a hundred thousand.
Rate limit and budget	A rate limit caps how much traffic arrives per unit of time, to protect capacity. A budget caps how much total is consumed or spent over a period, to protect money. They answer different questions and often use different units.
TPM and RPM	Tokens per minute and requests per minute: the two usual rate-limit units. LiteLLM documents both as separate limits (tpm_limit and rpm_limit).
max_tokens	A request parameter that caps how many tokens the model may generate. It is the only upper bound on output size that exists before the response does.
Reserve and reconcile	Admit a request by setting aside an estimate of its cost (input tokens plus max_tokens), then, once the response arrives, replace the estimate with the provider-reported usage and return the difference.
Tokenizer	The routine that turns text into tokens. Each model family has its own, so the same text can be a different number of tokens at different providers.
Time to first token (TTFT)	The wait before the first generated token arrives. A gateway adds its own delay on top of the provider's, which must stay small against it.
Where to read	LiteLLM: budgets and rate limits | Portkey: AI gateway | Cloudflare: AI Gateway

01 The request limit says 100%, finance says 130%

FRAMING

SCENARIO

A team has a budget of 1,000,000 tokens a day, which the gateway enforces with a limit of 500 requests a day and no per-request token cap. Today they sent 490 requests of 50 tokens and 10 requests that each carried a 128,000-token document. The dashboard shows 500 of 500 requests used.

HOW TO THINK ABOUT IT

When two measures of the same traffic disagree, work out which one tracks the thing you care about. Compute both on the same data and name what each one is blind to.

Q What does each accounting report?

A Requests: 500 of 500, so 100% of the quota, which looks normal. Tokens: 490 × 50 = 24,500, plus 10 × 128,000 = 1,280,000, for a total of 1,304,500, which is **130%** of the 1,000,000-token budget.

- The same traffic is exactly on budget by one measure and 30 points over it by the other.
- Ten requests, 2% of the count, produced 98% of the tokens.

Q Why is the request count blind to this?

A A request count has no fixed relationship to cost for LLM traffic. A request can carry 50 tokens or 128,000, and the second costs 2,560 times as much compute.

- Lesson 4 gives the same gap per minute: a limit of 100 requests a minute against a 128,000-token window allows up to 12,800,000 tokens a minute.

Q What do the real products count?

A Both, which is the point.

- LiteLLM has a tokens-per-minute limit and a requests-per-minute limit, and budgets in US dollars. Portkey limits requests or tokens and budgets by cost or tokens. Cloudflare AI Gateway's documented rate limit counts requests (docs, 2026-10-17).
- A product that counts only requests leaves the exposure in this question open, which is why token-aware limits were added to the others.

Q Could a per-request token cap have prevented it?

A Yes, by rejecting any request over, say, 20,000 tokens.

- It also rejects legitimate uses, such as summarizing an 80-page document in one call. An aggregate budget lets large requests through but needs real-time metering to enforce (Question 3).
- Many designs combine them: a generous per-request cap and an aggregate budget.

Q Reframe it.

A A limit must be in the unit that the cost is in. When the cost of a unit of traffic varies by orders of magnitude, counting units of traffic bounds nothing.

Basis: The 490 plus 10 request example and the 1,304,500 total are from the case study (illustrative); the 2,560 ratio and 12,800,000 are arithmetic (recomputed) and Lesson 4's example; product behavior is from their documentation (docs, 2026-10-17).

02 A token budget, or a dollar budget?

DESIGN CHOICE

SCENARIO

The platform team sets every team's budget as 1,000,000 tokens a day. The gateway routes by cost, so cheap traffic goes to a small model and hard questions go to a large one. A month later finance reports that one team spent 100 times as much as another that used the same number of tokens.

HOW TO THINK ABOUT IT

Ask whether the unit you limit is the unit you pay in. When prices vary across the things you route to, check whether the exchange rate between tokens and money is stable.

Q How can the same token count cost 100 times as much?

A Providers and models price tokens very differently. Suppose one model costs $0.15 per million tokens and another $15 (illustrative).

- 1,000,000 tokens on the first model cost $0.15. On the second they cost $15, a 100-times spread.
- Input and output tokens usually have different prices too, and output is typically the more expensive, so the mix of long prompts and long answers matters.

Q What does that say about the budget unit?

A A token budget bounds usage but not spend, once routing can send tokens to models with different prices. Spend needs a money budget.

- LiteLLM's budgets are in US dollars (max_budget) (docs, 2026-10-17). A dollar budget needs a price table per model and per token direction, kept current.
- Where one model serves all traffic, a token budget is a fine proxy.

Q So what are tokens still good for?

A Rate limits. They protect capacity and keep you inside each provider's own limits, which are set in tokens and requests per minute.

- Use two controls: tokens per minute (and requests per minute) for rate, money per period for budget.
- Report both to teams, so a surprise can be traced to volume or to model choice.

Q What goes wrong with the dollar figure if you are careless?

A Three things.

- Prices change, and a stale table mis-states spend.
- Providers tokenize differently, so the gateway's token count and the provider's billed count can differ. Reconcile against the usage the provider reports in its response.
- Cached or discounted input tokens are billed at a different rate. Use the provider's usage fields where they exist.

Q Reframe it.

A Limit what you pay in. Use tokens to protect capacity and money to protect the budget, and do not assume the exchange rate between them is constant.

Basis: The two prices and the 100-times spread are illustrative; the unit behavior of LiteLLM and Portkey is from their documentation (docs, 2026-10-17); arithmetic recomputed.

03 Admit a request before you know the answer

MECHANISM

SCENARIO

A request arrives with 2,000 input tokens and max_tokens set to 4,096. The gateway must decide now whether the caller's token bucket can afford it, but the model has not generated anything yet. Last month a team complained that the gateway rejects far more requests than its token limit implies.

HOW TO THINK ABOUT IT

Separate what is known at admission from what is not. Then design a protocol that is safe when the unknown is guessed wrong in either direction, and look at the cost of guessing high.

Q What does the gateway know, and what does it not?

A It knows the input exactly, by tokenizing the prompt. It does not know the output length. The only bound it has is max_tokens.

- Charging only the input at admission lets a request with a huge max_tokens through for free. Charging input plus max_tokens at admission is safe but pessimistic.

Q What is the protocol?

A Reserve, then reconcile.

- Reserve input plus max_tokens against the buckets before the call: 2,000 + 4,096 = 6,096 tokens.
- After the response, replace the reservation with the provider-reported usage and refund the difference. If the output was 300 tokens, actual use is 2,300 and the refund is 3,796.

Q Why does the team see too many rejections?

A Reservations overstate in-flight demand.

- 100 requests in flight reserve 609,600 tokens but use 230,000, 2.65 times as much.
- Under a 100,000-tokens-per-minute limit, reservation admits 100,000 ÷ 6,096 = 16 concurrent requests, while actual use would allow 100,000 ÷ 2,300 = 43.
- The cause is a generous default max_tokens. It throttles throughput and saves no tokens. Set max_tokens near the expected output length, or have the gateway cap it by route.

Q What about streaming, and a failed call?

A Streaming tokens arrive over time, so the gateway meters as they flow and can stop a stream when the reservation is used up.

- A failed call should release its reservation. A call that failed after the provider had started generating may still have been billed, so record it rather than assume nothing was spent.
- Always reconcile against the provider-reported usage, since providers tokenize differently.

Q Reframe it.

A When a decision must be made before a quantity is known, bound it, reserve the bound, and settle with the true value afterward.

Basis: The 2,000-token input, 4,096 max_tokens and 300-token output are illustrative; the reservation arithmetic is recomputed; reserve-and-reconcile is a design pattern (reasoned).

04 Is token counting too slow for a 20 ms budget?

LATENCY

SCENARIO

A reviewer objects to token-aware limits: “Tokenizing every request in the gateway adds latency. Count requests instead.” The design target is under 20 ms of gateway-added latency, against a model first-token time of 200 ms to 2 seconds.

HOW TO THINK ABOUT IT

Put the cost in the same unit as the target before arguing about it. Then look for what in the path really does eat the budget.

Q How much does tokenizing cost?

A A fast byte-pair-encoding tokenizer takes about 1 to 2 ms for a 2,000-token request (illustrative).

- Against a first-token time of 200 ms, 2 ms is 1%. Against 2 s, 1 ms is 0.05%. So the cost is between 0.05% and 1%.
- It is small, and it buys the ability to enforce the right limit.

Q What does the 20 ms budget get spent on?

A Everything the gateway does before forwarding.

- Tokenizing: 1 to 2 ms. Authentication and a routing decision: about a millisecond each.
- A counter operation against a shared store: one network round trip, perhaps 1 ms, for each of reserve and reconcile (illustrative).
- A cache lookup: an exact-match lookup is cheap. A semantic lookup must first embed the prompt, which can take tens of milliseconds and is the one item that can break the budget.

Q What follows for the semantic cache?

A It adds its cost to every miss, and misses are the common case. If the hit rate is 30%, 70% of requests pay for an embedding call that saved nothing.

- Measure the embedding latency, put it in the budget, and decide whether the hit rate pays for it. Run it in parallel with the call to the provider where the product allows.

Q How do you keep the budget honest?

A Measure the overhead directly: record the time between receiving a request and forwarding it, at p50 and p99.

- Alert on p99, since a small share of slow admissions are what users feel.

Q Reframe it.

A Do the arithmetic before the argument. The cost people worry about is often small, and the one nobody mentioned is the one that spends the budget.

Basis: Tokenizing time, counter round trip and the budget are illustrative; the percentages are arithmetic (recomputed); the budget breakdown is reasoned.

PART 2

Token buckets and limits

The algorithm behind the limit, how to size it, how to check several limits at once, and how several gateways share one.

READ THESE FIRST: CONCEPTS COVERED IN THIS PART

Fixed window	A counter that resets to zero at fixed intervals, such as every 60 seconds. A client can use the full limit just before a reset and again just after, so twice the limit passes in a moment.
Token bucket	A bucket holds up to a capacity of units and is refilled at a steady rate. Each request removes units. If there are not enough, it waits or is rejected. In any interval of length T at most capacity plus refill rate times T can pass.
Capacity and refill rate	Capacity is the largest burst the bucket allows. The refill rate is the long-run average. A bucket that refills once per day has refill rate equal to capacity divided by 86,400 seconds.
Burst	Traffic that arrives faster than the long-run rate for a short time. A bucket permits a burst up to its capacity.
Scope	What a limit applies to: an API key, a team or a model. A request can match several scopes and must satisfy all of them.
All-or-nothing deduction	When a request touches several buckets, either every bucket is charged or none is. Charging some and then failing leaks tokens.
Central counter store	A shared service (often a fast database) that holds the counters so that every gateway instance sees the same totals. It makes limits exact and becomes a dependency.
Fail open and fail closed	What the gateway does when it cannot check a limit: admit the request (open) or reject it (closed). Open keeps traffic flowing and risks overspend. Closed protects spend and risks an outage.
Where to read	LiteLLM: budgets and rate limits | Portkey: AI gateway

05 Why not just reset a counter every minute?

MECHANISM

SCENARIO

A colleague implements the rate limit as a counter of requests that resets at the start of every minute, with a limit of 100. Backends behind the gateway keep falling over at the top of the minute even though no key ever exceeds 100 requests a minute.

HOW TO THINK ABOUT IT

Look at what the rule allows across the reset. Then compare it with a rule that has no reset, using the same numbers.

Q What does the fixed window allow?

A A client can send 100 requests at 0:59 and 100 more at 1:00. Both minutes see 100, and 200 requests arrive within two seconds.

- The counter respects the rule, and the backend sees double the intended peak. In tokens it is worse: a 60,000-token limit lets 120,000 tokens in two seconds.

Q What does a token bucket do with the same traffic?

A Capacity 100, refilled at 100 ÷ 60 = 1.67 a second.

- Spending 100 at 0:59 empties it. By 1:01 only about 3 have refilled, so about 3 more pass, not 100.
- In any interval of length T the bucket passes at most capacity + rate × T: 100 + 1.67 × 2 = 103 in two seconds.

Q Why does the refill being continuous matter?

A There is no boundary to exploit. The allowed amount depends only on how much was spent recently, not on where the clock sits.

- The same idea works with LLM tokens as the unit, which is how the gateway's buckets are used (case study, Step 4).

Q Is there a case for the fixed window?

A It is simpler and cheap. It suits a quota that is defined per calendar period, such as a monthly allowance.

- Use the bucket for rate and the window counter for a per-period quota.

Q Reframe it.

A A rule with a reset has a seam. Check what happens on either side of it, and prefer a rule whose allowance depends on recent history instead of the clock.

Basis: The 100-per-minute example is illustrative; the bucket arithmetic is exact (recomputed); the algorithm is described in Step 4 of the case study.

06 Review the multi-scope limiter

CODE REVIEW

SCENARIO

Each request must pass three buckets: its API key, its team and the model it targets. A new function checks all three. In testing, teams report that their key budgets drain faster than their traffic explains, and under heavy load a few keys admit more than their limit.

```
def admit(buckets, cost):
    for b in buckets:                # key, team, model
        if b.tokens < cost:
            return False
        b.tokens -= cost
    return True
```

HOW TO THINK ABOUT IT

Trace the function with a request that passes the first bucket and fails the second. Then think about two requests arriving at the same instant.

Q What does the function do wrong when the second bucket is short?

A It deducts as it goes. The key bucket is charged, then the team bucket fails, the function returns False, and the key's tokens are gone though nothing was admitted.

- A team that is over its own limit therefore drains every key under it, and the keys' owners see budget vanish for requests that never ran.

Q What goes wrong under concurrency?

A Check and deduct are separate steps. Two requests can both see enough tokens and both deduct, so the bucket goes below zero and the limit is exceeded.

- The check and the deduction must be one atomic step: a lock, a transaction, or a single script on the counter store.

Q What is missing about time?

A The code never refills. A bucket's tokens must be topped up from the time elapsed since it was last touched, capped at its capacity, before every check.

- Without it, a bucket only ever shrinks.

Q State the corrected behavior.

A Refill every bucket, check that every one has enough, and only then deduct from all, as one atomic operation.

- If any bucket is short, nothing is deducted and the request is rejected or delayed, with the response saying which scope was short.

Q What would you test?

A Fail the second bucket and confirm the first is unchanged. Run many concurrent requests and confirm the total admitted never exceeds capacity plus refill. Advance the clock and confirm the refill is capped at capacity.

- Also test the reconcile step: reserve 6,096, settle at 2,300, and confirm the 3,796 returns to every bucket that was charged.

Q Reframe it.

A When one action changes several pieces of state, it must succeed or fail as a whole, and it must do so atomically.

REFERENCE CODE

```
def admit(buckets, cost, now):
    for b in buckets:
        b.refill(now)                # add rate * elapsed, capped at capacity
    if any(b.tokens < cost for b in buckets):
        return False                 # nothing deducted
    for b in buckets:
        b.tokens -= cost
    return True                      # run under one lock or one atomic script
```

Basis: The function is an illustrative sketch; the behavior is reasoned and was checked by hand, not run against a counter store.

07 Size the bucket

SIZING

SCENARIO

A key has a daily budget of 1,000,000 tokens. A new engineer proposes a bucket with capacity 1,000,000 and a refill of 1,000,000 ÷ 86,400 = 11.57 tokens a second. A teammate says that means no key can ever use more than a million tokens a day. Another worries a large request will never get through.

HOW TO THINK ABOUT IT

Both concerns are about how big the capacity is. Work out the worst case over a rolling day, and the smallest bucket that can admit your biggest legitimate request.

Q Is the teammate right that a key cannot exceed 1,000,000 a day?

A Not over a rolling day. A full bucket can pay 1,000,000 at once, then refill 11.57 × 86,400 = 1,000,000 more over the next 24 hours: **2,000,000** in one rolling day.

- A calendar-day quota would allow 1,000,000. The bucket removes the boundary burst but does not by itself give a hard daily cap.
- Check the refill arithmetic as well: 1,000,000 ÷ 86,400 = 11.574, and 5,000,000 and 20,000,000 give 57.87 and 231.48.

Q How do you get a hard cap?

A Use two controls. A smaller burst bucket limits short-term rate, and a separate counter for the period limits the total.

- For example: a bucket of 200,000 refilled at 11.57 a second, plus a daily counter that stops at 1,000,000.
- That limits a burst to 200,000 and a calendar day to 1,000,000.

Q Will a large request get through?

A Only if the bucket can hold it. A 128,000-token document with max_tokens 4,096 costs 132,096 tokens.

- A bucket with capacity below that rejects the request forever, however long the client waits. A 100,000-tokens-a-minute bucket with capacity 100,000 cannot admit it.
- So capacity must be at least the cost of the largest legitimate request, or large requests need their own route or limit.

Q What do you check in the config?

A Every capacity is at least 132,096. Every refill rate equals capacity divided by the period. The scopes nest sensibly: key at most team at most model.

- Here: key 1,000,000, team 5,000,000 and model 20,000,000 all clear 132,096, and 1,000,000 ≤ 5,000,000 ≤ 20,000,000.

Q Reframe it.

A A limit has a worst case you can compute. Work it out over the longest window you care about, and test it against the largest thing you must allow.

Basis: The numbers are the case study's illustrative configuration; the rolling-day worst case and the 132,096 cost are arithmetic (recomputed).

08 Four gateway instances, one limit

SCALE

SCENARIO

The gateway runs as four instances behind a load balancer. A team has a limit of 100,000 tokens a minute. The first design gives each instance 25,000 tokens a minute. Operators say some requests are rejected although the team is under its limit.

HOW TO THINK ABOUT IT

Ask whether the instances see equal shares of each team's traffic. Then compare the shared-store option by what it costs and what happens when it fails.

Q Why are requests rejected under the limit?

A Traffic is never perfectly even. Suppose one instance sees 60% of the team's demand, 60,000 tokens a minute, and the other three share the remaining 40,000.

- The busy instance admits 25,000 and rejects 35,000, 58% of its demand. The other three admit all 40,000.
- Total admitted is 65,000 of a 100,000 limit that exactly matched the demand. Thirty-five percent of legitimate traffic is rejected.

Q What does a central counter fix?

A Every instance deducts from the same bucket, so the limit is exact however traffic is spread.

- It costs a network round trip for each admission and each reconcile, perhaps 1 ms each, and it adds a dependency that must be available (Question 15).

Q Is there a middle path?

A Lease tokens. Each instance takes a chunk, for example 5,000 tokens, from the central bucket, spends it locally and asks for more when it runs low.

- The store sees far fewer operations. The worst overshoot is bounded by the leases outstanding: with four instances and 5,000-token leases, at most 20,000 tokens are held out.
- Return unused leases on a timer, or tokens leased to a stalled instance are lost.

Q How do you choose?

A By how exact the limit must be and how bad an outage of the store is.

- Hard money budgets favor the central store, with fail-closed behavior. Soft per-minute rate limits can tolerate local counters or leases.

Q Reframe it.

A Splitting a shared limit across servers assumes the load splits the same way. Measure how it actually splits, then choose between exactness and independence.

Basis: The four instances and the 60/40 split are illustrative; the rejection arithmetic and lease bound are recomputed; the options are design arguments (reasoned).

PART 3

Routing, fallback and caching

Choosing a provider for each request, surviving a failing one without making it worse, and caching without serving the wrong answer.

READ THESE FIRST: CONCEPTS COVERED IN THIS PART

Routing strategy	The rule that picks a provider or model for a request. Common ones: cost-based (cheapest that qualifies), capability-based (has the feature or context window needed), latency-based (fastest recently), least-busy, and weighted random shuffle.
Capability floor	A hard filter applied before any preference: only providers that can serve the request at all, for example with a large enough context window or function calling, stay in the candidate set.
Health check and cooldown	The gateway marks a provider unhealthy after enough failures and stops sending it traffic for a cooldown period. LiteLLM documents a default of 3 allowed failures and a 5-second cooldown.
Retry with backoff	Trying a failed request again after waiting, with the wait growing each time (exponential backoff), usually with some random jitter so many clients do not retry at the same instant.
Retry budget	A cap on retries as a share of total traffic, for example 10%, so that retries cannot multiply the load on a provider that is already struggling.
Exact-match cache	A cache keyed on the literal request: model, full prompt and parameters. A hit means an identical request was seen before.
Semantic cache	A cache keyed on an embedding of the prompt, matching on similarity above a threshold. It hits on near-duplicate prompts, which can mean a different question.
Cache key and scope	What a cached answer is stored under. Including the tenant in the key stops one tenant receiving another's answer. Including the parameters stops a stale or differently sampled answer being reused.
Where to read	LiteLLM: routing, retries and cooldowns | Portkey: AI gateway | Cloudflare: AI Gateway

09 Pick a routing strategy

DESIGN CHOICE

SCENARIO

The gateway fronts three providers. A is a large model with a 128,000-token context, expensive. B is a small, cheap model with a 32,000-token context. C is a mid-tier model with function calling. Product wants the lowest cost, and users want the best answers. An engineer proposes pure cost-based routing.

HOW TO THINK ABOUT IT

Separate hard constraints from preferences. Apply constraints as filters first, then choose among what is left, and check each preference for the way it fails when used alone.

Q What comes before any strategy?

A A capability filter. Only providers that can serve the request at all stay in the candidate set.

- A request with a 60,000-token prompt cannot go to B, however cheap. A request that needs function calling cannot go to a model without it.
- These are requirements, not preferences, and routing on cost or speed among providers that cannot answer is an error.

Q Why is pure cost-based routing a problem?

A It sends every request to the cheapest provider that passes the filter, including quality-sensitive ones that need a stronger model.

- Add a capability or quality tier to the request, and let cost choose only within the tier.

Q Why is pure latency-based routing a problem?

A It concentrates traffic. If A has a median of 300 ms and B 340 ms, every request goes to A.

- A has its own limits. At 600,000 tokens a minute of demand against a 500,000 limit, the extra traffic gets rate-limit errors.
- It also undermines the reason for having several providers. LiteLLM's latency-based strategy has a buffer setting that spreads traffic among providers whose latencies are close (docs, 2026-10-17).

Q What do real gateways default to?

A LiteLLM's documented default is simple-shuffle, weighted or random, and its documentation recommends it for production. It calls usage-based routing not recommended for production because of the overhead of the shared store it needs (docs, 2026-10-17).

- That is a hint: simple, stateless choices are robust, and each strategy that needs shared state adds a dependency.

Q How do you decide between the options?

A Replay real traffic through each and compare cost, p99 latency, error share and quality on a labeled sample.

- Measure per route, since an average hides a strategy that works for one kind of request and fails another.

Q Reframe it.

A Apply requirements as filters and preferences as scores. A preference used alone will eventually pick something that cannot do the job.

Basis: Provider characteristics and numbers are illustrative; the strategy list and the production recommendation are from LiteLLM's documentation (docs, 2026-10-17); the approach is reasoned.

10 A provider keeps flapping

FALLBACK

SCENARIO

The gateway marks a provider unhealthy when more than 5% of its requests in the last 60 seconds return a 5xx error. At night, with about 10 requests a minute, the provider is marked unhealthy and healthy again every few minutes, and during the day a failing provider seems to get more traffic, not less.

HOW TO THINK ABOUT IT

A threshold on a rate means nothing without a sample size. Then look at what the gateway does after the trip: how long it waits, and what it does to the providers that are left.

Q Why does the rule misfire at night?

A With 10 requests in the window, one error is 10%, which is above 5%.

- Require a minimum number of requests, for example 20, before the rule can trip. At 20 requests, one error is exactly 5% and does not trip it; two errors, 10%, do.

Q What stops the flapping?

A Hysteresis: a cooldown after a trip, then a cautious return.

- LiteLLM documents a cooldown (default 5 seconds), three allowed failures by default, an immediate cooldown on a 429, and cooldown when more than 50% of requests in the current minute fail (docs, 2026-10-17). The 5% rule here is this case study's illustration, not that product's rule.
- After the cooldown, send a few probe requests or a small share of traffic, and restore full traffic only if they succeed.

Q Why does a failing provider seem to get more traffic?

A Retries. If each failing request is retried 3 times, a provider already in trouble sees up to 4 times the load, and so do the fallbacks.

- Cap retries as a share of traffic, for example 10%, so the extra load is at most 1.1 times. Use exponential backoff with jitter. LiteLLM retries rate-limit errors with exponential backoff and other errors immediately (docs, 2026-10-17).

Q What would you watch?

A Fallback trigger frequency per provider, retry share of traffic, and the time providers spend in cooldown.

- A rising trend over weeks is a signal to deprioritize the provider, independent of any single incident.

Q Reframe it.

A A health rule needs a minimum sample, a cooldown and a retry limit. Without all three, the protection becomes a source of instability.

Basis: The 5% rule, the 60-second window and the 10 requests a minute are illustrative; the cooldown and retry behaviors are from LiteLLM's documentation (docs, 2026-10-17); the amplification arithmetic is recomputed.

11 Which errors should the gateway retry?

RELIABILITY

SCENARIO

A provider call fails. The gateway currently retries every failure on the next provider. Teams complain about duplicate charges, and a request that violates a content policy is being sent to three providers in a row.

HOW TO THINK ABOUT IT

Sort failures by whether another attempt could succeed. Then ask what a retry costs when the first attempt may have already done work.

Q Which failures can another attempt fix?

A Transient ones: timeouts, connection resets, 5xx server errors, and rate-limit (429) responses, the last after a backoff.

- These may succeed later or elsewhere.

Q Which cannot?

A Failures that are about the request itself.

- Bad requests (400), authentication failures (401 and 403) and unknown models will fail on every attempt.
- A content-policy rejection will usually be repeated. A context-window overflow will fail at any provider with the same window, and the fix is capability routing to a larger one, not a retry.
- LiteLLM's documentation excludes context-window and content-policy errors from failover (docs, 2026-10-17).

Q Why the duplicate charges?

A A timeout does not mean nothing happened. The provider may have started generating, and may bill for it.

- A 2,300-token request that times out and is retried may be billed twice, 4,600 tokens.
- Use timeouts sized to the expected time, separate the wait for the first token from the total time, and record the usage of failed attempts.

Q What about streaming?

A After the first token has reached the client, the gateway cannot switch providers without restarting the answer.

- Fall back only before the first token. After that, end the stream with a clear error or let the client retry.

Q Reframe it.

A Retry only what a retry can fix, and remember that a failure may have cost something already.

Basis: The classification is reasoned; the context-window and content-policy exclusion is from LiteLLM's documentation (docs, 2026-10-17); the 2,300-token example is illustrative.

12 The semantic cache answered the wrong question

CACHING

SCENARIO

A support assistant uses a semantic cache with a similarity threshold. A customer who asked to cancel order 1243 was told that order 1234 had been cancelled, an answer cached for someone else. Cache hit rate is 40%, and the team likes the savings.

HOW TO THINK ABOUT IT

A cache that matches on meaning can match on a meaning that is too coarse. Find what the match ignores, where it must not be used, and how to measure its error against its saving.

Q What happened?

A The two prompts differ in one digit, so their embeddings are almost identical, and the similarity score passed the threshold.

- The embedding captures what the sentence is about, not exact identifiers. The cached answer was right for a different order.

Q Where must a semantic cache not be used?

A Wherever the exact details matter.

- Prompts that contain identifiers, amounts, dates or personal data, and requests that trigger actions.
- Those can use an exact-match cache, which hits only on identical requests and cannot confuse two orders. A semantic cache fits FAQ-style traffic, where near-duplicate questions deserve the same answer.

Q How do you set the threshold?

A With data. Build labeled pairs of prompts marked same answer or different answer, and measure the hit rate and the wrong-hit rate at each threshold.

- Suppose the cache gives 4,000 hits per 10,000 requests, and 2% of hits are wrong: 80 wrong answers per 10,000 requests, against 4,000 × 2,300 = 9.2 million tokens saved (illustrative).
- Whether 80 wrong answers is acceptable depends on the product. For an order system it is not.

Q What else belongs in the cache key?

A The tenant, so one customer's answer is never served to another, and the model and parameters, so an answer made under different settings is not reused.

- A semantic hit may also come from a provider that routing would not choose now. Decide whether that is acceptable.

Q Reframe it.

A A cache trades exactness for hit rate. Know what the match ignores, measure how often that is wrong, and keep it away from cases where the ignored detail is the whole question.

Basis: The order numbers, the 40% hit rate and the 2% wrong-hit rate are illustrative; the arithmetic is recomputed; the cache design points are from Step 4 of the case study (reasoned).

13 The hit rate collapsed after a release

DEBUGGING

SCENARIO

Cache hit rate was 35% for months. After last week's release it is 5%, and the provider bill has risen. Nobody changed the cache.

HOW TO THINK ABOUT IT

A cache is as good as its key. Look for what the release put into the request that is different every time, and put a price on the loss.

Q What do you check first?

A Split the hit rate into exact and semantic, then compare the request templates before and after the release.

- If exact-match hits fell and semantic hits did not, something in the key changed. Diff the prompt template, the parameters and the headers that go into the key.

Q What is the usual cause?

A A value that changes on every request ended up in the prompt: a timestamp, a request id, a user name placed at the top.

- The meaningful part of the prompt did not change, but two requests are no longer byte-for-byte identical, so the exact-match cache never hits.

Q What did it cost?

A Provider calls go from 65% of requests to 95%: 0.95 ÷ 0.65 = 1.46, so provider tokens and spend rose about 46%.

- A fall from 35% to 5% in hit rate looks like a 30-point change and is a 46% increase in cost.

Q How do you fix it, and what else lowers hits?

A Move the variable parts to the end of the prompt or out of the cached prefix, normalize whitespace and ordering, and keep volatile fields out of the key.

- Scoping the cache per tenant also lowers hits. If 20 tenants send the same public prompt, a shared cache misses once and hits 19 times, 95%. Isolated caches miss 20 times. Share only what is public.

Q How do you catch it sooner?

A Alert on the hit rate by template version and on a drop of more than a few points, and run the key-generation code in tests with realistic requests.

> **Q Reframe it.**
>
> **A** A cache key must contain exactly what makes two requests the same. Anything extra that varies silently destroys the hit rate.

Basis: The 35% and 5% hit rates and the 20 tenants are illustrative; the 1.46 ratio and 95% are arithmetic (recomputed); the diagnosis follows Step 6 of the case study (reasoned).

PART 4

Architecture, failure modes and safety

What the gateway does to availability, what happens when its state fails, how streaming interacts with budgets, what ten times the traffic breaks, and what to review for security.

READ THESE FIRST: CONCEPTS COVERED IN THIS PART

Availability and downtime	Availability is the share of time a service works. 99.9% is 8.76 hours of downtime a year. Components in series multiply their availabilities. A redundant pair fails only when both parts fail at once.
Active-active	Two or more instances all serving traffic at the same time, so losing one loses only part of the capacity. It needs the instances to share or reconcile state.
Correlated failure	Several components failing from one cause, such as one configuration push, one dependency or one region. The independence assumed in availability arithmetic does not hold for them.
Streaming (server-sent events)	The provider sends the answer in pieces as it is generated. A gateway forwards the pieces as they arrive and can meter tokens as they pass.
Cancellation	When a client disconnects, the gateway tells the provider to stop generating, so tokens are not produced and billed for nobody.
Denial of wallet	An attack or accident that drives up spend, for example a leaked key used to send large volumes, rather than taking the service down.
Tenant isolation	The guarantee that one customer's data, cached answers and limits cannot affect or reveal another's.
Secret manager	A service that stores credentials such as provider API keys, controls who can read them and supports rotation, so they are not kept in code or configuration files.
Where to read	LiteLLM: budgets and rate limits | LiteLLM: routing, retries and cooldowns | Cloudflare: AI Gateway

14 Does adding a gateway improve availability?

AVAILABILITY

SCENARIO

Calling one provider directly has 99.9% availability. The team wants a gateway with a second provider for fallback. The gateway itself is 99.95% available. Management asks whether this makes the service more or less reliable.

HOW TO THINK ABOUT IT

Draw the chain and multiply. Then ask what the formula assumes about how the parts fail together.

Q What is the baseline?

A Direct to one provider: 0.999, which is 8.76 hours of downtime a year.

Q What does one gateway change?

A It depends on whether there is a second provider.

- Gateway and one provider in series: 0.9995 × 0.999 = 0.9985, 13.1 hours a year. That is worse than calling the provider directly.
- Gateway and two providers with fallback: 0.9995 × (1 − 0.001²) = 0.999499, 4.4 hours a year. That halves the downtime.

Q What caps the benefit?

A The gateway. Its own 0.05% unavailability is 4.4 hours a year, which is almost all of the chain's downtime.

- The providers pair to 99.9999%, so they contribute almost nothing. The gateway is now the weakest link, the single point of failure the design was supposed to remove, moved up one layer.

Q What does a gateway pair give?

A $(1 - 0.0005^2) \times (1 - 0.001^2) = 0.9999988$, about 39 seconds of downtime a year.

- Run the pair active-active, ideally across regions, so a regional failure is also covered.

Q What does the formula leave out?

A It assumes independent failures.

- One bad configuration pushed to both gateways at once takes both down. So do shared dependencies: the counter store, DNS, the same cloud region or the same certificate expiry.
- Use staged rollouts and a canary for gateway configuration. Failover also takes time: DNS-level failover is bounded by the record's time to live (Lesson 4).

Q Reframe it.

A Redundancy only helps where failures are independent. Compute the benefit, find the new weakest link, and ask what could take out both copies at once.

Basis: Component availabilities are illustrative; the series and parallel formulas are from Lesson 3, Concept 3; all figures are arithmetic (recomputed).

15 The counter store is down

FAILURE MODE

SCENARIO

All budget and rate counters live in one shared store. It becomes unavailable for 30 minutes. The gateway has to choose: keep admitting requests or reject them. A client with a bug is sending 100 requests a second, each 2,300 tokens, on a key with a 1,000,000-token daily budget.

HOW TO THINK ABOUT IT

Price each failure mode before choosing. Then look for a third option between all-open and all-closed.

Q What does failing open cost here?

A Nothing caps the runaway client. 100 requests a second × 2,300 tokens = 230,000 tokens a second, and over 1,800 seconds that is **414,000,000 tokens**, 414 times the daily budget.

- Money is spent before anyone sees it. LiteLLM's budgets fail open if no database is configured (docs, 2026-10-17).

Q What does failing closed cost?

A Every team's traffic is rejected for 30 minutes: an outage of the gateway caused by a counter store.

- LiteLLM documents a fail-closed rate-limiting option that returns 503 (docs, 2026-10-17).

Q Is there a middle path?

A Yes: degrade to conservative local limits.

- Each gateway instance keeps a local cap per key, chosen low, that applies only while the store is unreachable. The runaway client is bounded, and ordinary traffic still flows.
- Time-limit the degraded mode and alert at once, so it is not mistaken for normal operation.

Q How do you choose?

A By limit type.

- Hard money budgets favor failing closed or degrading, since overspend is hard to undo. Soft rate limits can fail open for a short time with an alert.
- Decide it in advance and write it down, so the choice is not made by the first engineer paged at 3 a.m.

Q How do you know it works?

A Test it. Take the store down in a game day and watch what the gateway does, what the runaway client spends and how fast the alert fires.

Q Reframe it.

A A safety control has a failure mode, and the failure mode is a design decision. Choose it per risk, test it, and do not leave it as a default.

Basis: The 30-minute outage, the 100 requests a second and the 2,300-token request are illustrative; the 414,000,000 and 414 times are arithmetic (recomputed); product behaviors are from LiteLLM's documentation (docs, 2026-10-17).

16 Streaming and the budget

MECHANISM

SCENARIO

Most requests stream their answers. A product owner asks three things: how the gateway can meter a stream, what happens when a client closes the tab midway, and what happens when a team's budget runs out in the middle of an answer.

HOW TO THINK ABOUT IT

Treat the stream as a sequence of small charges that can stop at any point. Then decide, for each way it can stop, who pays and who sees what.

Q How is a stream metered?

A The gateway counts tokens as they pass and reconciles against the provider's reported usage when the stream ends.

- Check how each provider reports usage for a stream, since it may be in a final message and may need to be requested. Where it is missing, the gateway's own count is the estimate.
- If the gateway buffers the whole answer to cache it, the user loses the benefit of streaming. Forward each piece immediately and keep a copy on the side.

Q What happens if the client disconnects?

A The gateway should cancel the upstream request.

- Otherwise the provider keeps generating tokens nobody will read. With max_tokens 4,096 and a disconnect after 300 tokens, cancelling avoids up to 3,796 generated tokens, all billed, and returns the unused reservation (Question 3).

Q What if the budget runs out mid-answer?

A There are two honest options.

- Cut the stream. The cost is bounded and the user gets a truncated answer.
- Let it finish and charge the overage. The user is not interrupted, and spend can exceed the budget by at most one answer.
- Choose by tier: let interactive users finish, and cut batch jobs. Because the reservation already covers max_tokens, the overage is usually zero.

Q What about a stream that fails halfway?

A The client has already received part of an answer. Do not silently retry on another provider and append its text.

- End the stream with a clear error, record the tokens already generated, and let the client decide whether to retry.

Q Reframe it.

A A long-running response is a series of decisions, not one. Define what happens at every point where it can stop.

Basis: The 4,096 max_tokens and 300-token disconnect are illustrative; arithmetic recomputed; streaming behavior follows Lesson 10, Concept 3, and the case study's Step 5 (reasoned).

17 Traffic grows tenfold

SCALE

SCENARIO

Peak traffic is 200 requests a second at 2,300 tokens each, with about 2 ms of gateway CPU per request. It is expected to grow tenfold. Your manager asks which part of the gateway breaks first.

HOW TO THINK ABOUT IT

Scale each quantity by ten and compare with its limit. The part with the smallest margin, often something outside the gateway, is the one to plan for.

Q What does ten times do to gateway CPU?

A 200 × 2 ms = 0.4 cores today. At 2,000 requests a second, 4 cores.

- Not the first problem. Provision for the active-active pool and for headroom; the CPU itself is small.

Q What does it do to the counter store?

A Each request does about 6 counter operations: reserve and reconcile on three buckets. That is 1,200 operations a second today and 12,000 at ten times.

- Compare 12,000 a second, plus the round trip each adds to every request, with the store's measured capacity. Leasing tokens (Question 8) cuts the operation count.

Q What does it do to the providers?

A Demand is 200 × 2,300 = 460,000 tokens a second, 27.6 million a minute. At ten times it is 276 million a minute.

- Providers have their own token-per-minute limits. Spread traffic across keys and providers, and ask for higher limits well ahead.
- Price follows volume, so the budget is a limit of its own.

Q What does it do to the cache?

A The same hit rate saves ten times the tokens, so the cache gets more valuable. Check memory and eviction, and keep tenant scoping.

Q How do you degrade before refusing?

A In tiers, in order of least harm, as Lesson 10 describes: cap output length, shorten context, route a share of traffic to a cheaper model, and only then reject.

- Tell callers what happened with a clear status, so they back off and do not retry harder.

Q Reframe it.

A Growth moves the bottleneck. Scale every quantity and find the one whose limit you do not control.

Basis: The 200 requests a second, 2,300 tokens and 2 ms of CPU are illustrative; the scaling figures are arithmetic (recomputed); the degradation tiers are from Lesson 10, Concept 2.

18 Security review of the gateway

SECURITY

SCENARIO

A security reviewer is looking at the gateway before launch. It holds the keys to every provider, caches responses, logs prompts and sits in front of every team. They ask for a threat list and a mitigation for each item.

HOW TO THINK ABOUT IT

A gateway concentrates risk by design: one place holds the credentials, the logs and the shared state. List what an attacker, or a mistake, gains from each concentration.

Q What is the risk of holding all provider keys?

A A single place to steal. Anyone who reads the gateway's configuration can spend every team's money.

- Keep keys in a secret manager, never in code or configuration files. Scope them narrowly and rotate them (Lesson 8, Concept 3).

Q How can caching leak data?

A A cache whose key lacks the tenant returns one caller's answer to another.

- Put the tenant in the key. A poisoned entry is served to everyone who matches it, so limit who can write to the cache and expire entries.
- Keep semantic caching away from personal data (Question 12).

Q What is denial of wallet, and how do you bound it?

A A leaked or abused key burns the budget without taking anything down.

- Per-key limits, burn-rate alerts that project the day's total (60% used by noon is on course for 120%) and a hard cap for keys of unknown ownership.

Q Where does the tenant identity come from?

A From the authenticated key, never from a header the caller can set. A caller that can name its own tenant can read another tenant's cache and spend another's budget.

Q What about logs and prompt injection?

A Logs hold prompts and responses, which contain personal and confidential content. Apply retention and access rules (Lesson 8, Concept 5).

- The gateway does not stop prompt injection. Budgets and scoped keys limit the damage, but defense belongs to the application (Lesson 8, Concept 1).

Q Reframe it.

A Wherever a design concentrates something, a credential, state or logs, it concentrates the attack. List the concentrations and defend each.

Basis: The threat list follows Step 6 of the case study (reasoned); the 60% by noon example is illustrative; nothing here was tested against an attack.

Sources and confidence

The case study this book is built on is the AI System Design Primer's Capstone #15. No company has published a primary engineering write-up of this pattern, so the sources are the products' documentation: two LiteLLM pages, and the overview pages of Portkey's and Cloudflare's gateways, all read on 2026-10-17. Overview pages state what a feature is, not how it performs, so no performance claim is made. Nothing described here was run on real systems, and scenario figures are illustrative.

- LiteLLM documentation: budgets and rate limits (tpm_limit, rpm_limit, max_budget, budget_duration), read 2026-10-17
https://docs.litellm.ai/docs/proxy/users
- LiteLLM documentation: router (routing strategies, retries, cooldowns, failover exclusions), read 2026-10-17
https://docs.litellm.ai/docs/routing
- Portkey documentation: AI gateway (fallbacks, load balancing, caching, budget and rate limits), overview page read 2026-10-17
https://portkey.ai/docs/product/ai-gateway
- Cloudflare AI Gateway documentation (caching, request-count rate limiting, analytics, retry and fallback), overview page read 2026-10-17
https://developers.cloudflare.com/ai-gateway/